INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Tema de la guía práctica proporcionada por el docente
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Nivel - Paralelo
Alumnos participantes:	Sánchez Lema Isaac Adrián Bravo López Jordan Samuel ……..
Asignatura:	Algoritmos y Lógica de la Programación
Docente:	Ing. José Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:
Desarrollar un sistema de gestión de videojuegos académico en C++ mediante el paradigma de **Programación Orientada a Objetos (POO)**, que integre lógica de ranking, un módulo interactivo de ahorcado y control de versiones en GitHub, asegurando la optimización de procesos a través de algoritmos recursivos y la correcta depuración de estructuras condicionales.

Específicos:

- **Implementar una arquitectura modular** mediante el uso de clases (Jugador, Partida, Ahorcado) con encapsulamiento estricto (atributos privados y métodos de acceso), permitiendo una gestión organizada de los puntajes, misiones y penalizaciones del videojuego.
- **Optimizar el cálculo de bonificaciones** mediante la aplicación de funciones recursivas con utilidad real, además de garantizar la integridad del sistema a través de la corrección de fragmentos de código defectuosos y la validación de entrada de datos.
- **Gestionar el ciclo de vida del desarrollo** utilizando GitHub Desktop para el versionamiento del código, evidenciando un progreso incremental de al menos cuatro etapas significativas (commits) y documentando la metodología en un repositorio remoto profesional.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 3 horas
No presenciales: 0 horas

2.4 Instrucciones

Lógica y análisis del problema

- identificar entradas, procesos, salidas y reglas de negocio
- convertir un enunciado en casos de uso y validaciones
- elaborar al menos una prueba de escritorio antes de programar

Fundamentos de C++

- tipos de datos, variables, constantes y operadores
- lectura y escritura con cin y cout
- prioridad de operadores y expresiones lógicas

Estructuras de control

- if, if-else, else if y switch
- for, while y do-while
- menús validados y repetición controlada

Funciones

- funciones para registrar, validar, buscar, ordenar y reportar
- paso por valor y paso por referencia
- devolver correctamente resultados y evitar errores lógicos

Arreglos o vectores

- almacenar varios registros del sistema
- recorrer, contar, acumular y generar reportes
- manejar información del caso principal y del Ahorcado

Programación orientada a objetos

- clase, objeto, atributo y método
- encapsulamiento con atributos privados
- constructores, getters y setters

Búsqueda y ordenamiento manual

- búsqueda secuencial
- ordenamiento burbuja, selección o inserción
- ordenar por uno o dos criterios sin usar sort

Recursividad

- usar una función recursiva útil dentro del sistema
- por ejemplo: sumar multas, totalizar ventas o contar registros críticos

Depuración

- corregir errores como = en lugar de ==
- detectar ciclos infinitos, retornos incorrectos o condiciones mal planteadas
- justificar técnicamente la corrección realizada

Módulo Ahorcado

- palabra oculta, intentos, letras usadas y validación
- niveles de dificultad, puntuación e historial
- adaptación del vocabulario al grupo o caso asignado

GitHub Desktop y GitHub

- crear repositorio local
- realizar commits significativos durante el desarrollo
- publicar el repositorio y redactar un README mínimo

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

-

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☐Plataformas educativas
☐Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☒Recursos audiovisuales
☐Gamificación
☒Inteligencia Artificial
Otros (Especifique): _____

2.6 Actividades por desarrollar

Videojuego académico con ranking y penalizaciones

Contexto del caso: Evaluar jugadores según puntos, errores, tiempo, bonificaciones, vidas perdidas y ranking final.

Clases mínimas sugeridas: Jugador y Partida

Requerimientos funcionales obligatorios

- Registrar jugadores y partidas con puntos de misión 1, 2 y 3, errores, tiempo y eventos especiales.
- Calcular puntaje base, bonificaciones y penalizaciones.
- Asignar rango y aplicar restricciones por vidas o errores.
- Buscar por alias y rango, ordenar manualmente el ranking por puntaje y tiempo.
- Generar top 3, promedio de errores por rango e inconsistencias.

Integración obligatoria con Ahorcado: Construir un Ahorcado con misiones, rangos, objetos especiales, bonificaciones y penalizaciones.

Fragmento defectuoso para depurar y justificar

```
int rangoPorPuntaje(int puntaje){
  if(puntaje >= 900) return 5;
  else if(puntaje >= 700) return 4;
  else if(puntaje >= 500) return 3;
  else if(puntaje >= 300) return 2;
  else if(puntaje < 300) return 1;
  else return 0;
}
```

El estudiante debe corregir el fragmento, justificar los errores encontrados y demostrar la corrección con una prueba mínima.

a) ANÁLISIS DEL PROBLEMA

El problema central de este proyecto radica en la necesidad de procesar múltiples variables (puntajes de 3 misiones, errores cometidos y tiempo empleado) para asignar un **rango de competencia** preciso, enfrentando los siguientes retos:

- La falta de un sistema de penalizaciones y bonificaciones dinámicas.

- La ausencia de un ranking organizado que permita identificar a los mejores perfiles.
- La necesidad de incluir métodos de aprendizaje lúdicos (Ahorcado) con terminología técnica del dominio.

Para resolver esta problemática, el sistema debe gestionar el flujo de información de la siguiente manera:

- **Entradas (Inputs):**
  - Datos de identificación (Alias del jugador).
  - Métricas de rendimiento (Puntos por misiones M_1, M_2, M_3).
  - Variables de control (Número de errores y tiempo total en segundos).
- **Procesamiento (Cálculos Técnicos):**
  - **Puntaje Base:** $\sum (M_n)$.
  - **Bonificación:** Calculada mediante un algoritmo **recursivo** basado en el tiempo ahorrado.
  - **Penalización:** Factor multiplicador basado en la cantidad de errores.
  - **Clasificación:** Lógica condicional para asignar rangos del 1 al 5 (corrigiendo redundancias previas).
- **Salidas (Outputs):**
  - Reporte detallado por jugador con puntaje final y rango.
  - Ranking ordenado de mayor a menor puntaje (Top 3).
  - Interfaz interactiva del juego de Ahorcado.

El problema no solo es funcional, sino que exige el cumplimiento de estándares de ingeniería de software:

1. **Modularidad:** El código no puede ser un solo bloque; debe estar dividido en archivos de cabecera (.h) e implementación (.cpp) para asegurar el mantenimiento.
2. **Encapsulamiento:** Los datos sensibles deben estar protegidos (privados), permitiendo el acceso solo mediante interfaces controladas (getters/setters).
3. **Trazabilidad:** Se requiere un historial de cambios en **GitHub Desktop** que demuestre el desarrollo incremental y el control de versiones, evitando la pérdida de información durante la fase de construcción.

La solución propuesta consiste en una aplicación de consola robusta programada en **C++**. Esta aplicación utiliza una arquitectura orientada a objetos donde la clase Jugador actúa como el controlador de la lógica, apoyándose en la clase Partida para el almacenamiento de datos. Se integra la **recursividad** como método de optimización de cálculo y un algoritmo de **ordenamiento manual** (Bubble Sort) para gestionar el ranking sin depender de librerías externas de alto nivel, cumpliendo así con el rigor académico del examen.

b) VARIABLES DE ENTRADA, PROCESO Y SALIDA

Tabla 1. Tabla de variables de entrada

Variable	Tipo de Dato	Ubicación	Descripción
op	int	main.cpp	Opción seleccionada del menú principal.

nombre	string	main.cpp	El alias o nombre del jugador para el registro.
m1, m2, m3	int	main.cpp	Puntos obtenidos en cada una de las 3 misiones.
err	int	main.cpp	Cantidad de errores o eventos fallidos registrados.fallidos registrados.
tiempo	int	main.cpp	Tiempo total empleado en segundos.
letra	char	Ahorcado.cpp	Carácter ingresado para intentar adivinar la palabra.

Tabla 2. Tabla de variables de salida

Variable	Tipo de Dato	Ubicación	Descripción
puntajeFinal	int	Jugador.cpp	Resultado del cálculo: (Base + Bonos)- Penalizaciones.
rango	int	Jugador.cpp	Nivel asignado (1 a 5) tras evaluar el puntaje final.
ranking	vector<Jugador>	main.cpp	Lista ordenada (Top 3) de los mejores puntajes.
palabraOculta	string	Ahorcado.cpp	Estado actual de la palabra
intentosRestantes	int	Ahorcado.cpp	Contador de vidas en el juego del ahorcado.

c) DIAGRAMA DE FLUJO (PSEINT)

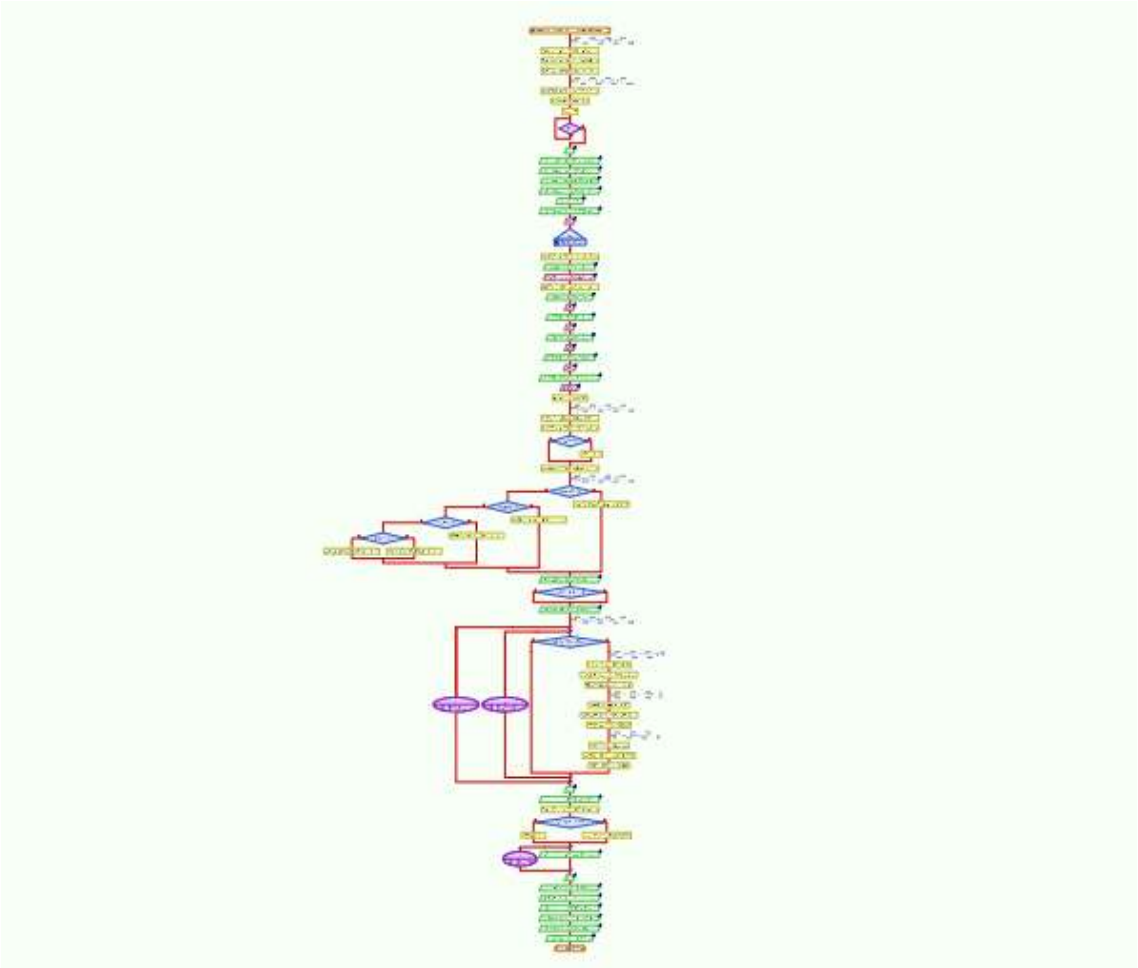


Ilustración 1. Digrama de flujo

d) CÓDIGO PSEINT

```
Algoritmo Sistema_Gestion_Videojuegos_GrupoK

   // 1. DEFINICIÓN DE VARIABLES

   Definir op, numJugadores, i, j Como Entero

   Definir nombres, tempN Como Cadena

   Definir puntajes, rangos, tempP, tempR Como Entero



   // 2. DIMENSIONAMIENTO (Capacidad para 100 registros)

   Dimension nombres[100], puntajes[100], rangos[100]



   numJugadores <- 0
```

```
op <- 0

Mientras op <> 4 Hacer
    FinMientras

  Escribir ""
  Escribir "--- SISTEMA DE GESTION: GRUPO K ---"
  Escribir "1. Registrar Jugador y Partida"
  Escribir "2. Ver Ranking (Top 3)"
  Escribir "3. Modulo Ahorcado Academico"
  Escribir "4. Salir"
  Escribir "Seleccione una opcion: "
  Leer op

  Segun op Hacer
          FinSegun

      numJugadores <- numJugadores + 1
      Escribir "Alias del jugador: "
      Leer nombres[numJugadores]

      Definir m1, m2, m3, err, tiempo, base, bono, total Como Entero
      Escribir "Puntos Mision 1: "; Leer m1
      Escribir "Puntos Mision 2: "; Leer m2
```

```
Escribir "Puntos Mision 3: "; Leer m3

Escribir "Errores cometidos: "; Leer err

Escribir "Tiempo empleado (seg): "; Leer tiempo


base <- m1 + m2 + m3

// Llamada a la funcion recursiva

bono <- CalcularBonificacion(120 - tiempo)

total <- base + bono - (err * 15)


Si total < 0 Entonces total <- 0

                        FinSi


puntajes[numJugadores] <- total


// Lógica de Rangos (Fase de Depuración - Sin redundancias)

Si total >= 900 Entonces

   rangos[numJugadores] <- 5

Sino

   Si total >= 700 Entonces

      rangos[numJugadores] <- 4

   Sino

      Si total >= 500 Entonces

         rangos[numJugadores] <- 3

      Sino

         Si total >= 300 Entonces
```

```
                rangos[numJugadores] <- 2
            Sino
                rangos[numJugadores] <- 1
            FinSi
        FinSi
    FinSi
FinSi
```

e) Código en C++

```
// --- Partida.h ---
#ifndef PARTIDA_H
#define PARTIDA_H


class Partida {
private:
    int misiones[3];
    int errores;
    int tiempo;


public:
    Partida();
    void setDatos(int mision1, int mision2, int mision3, int eventos, int tiempo);
    int getPuntajeBase() const;
    int getErrores() const;
    int getTiempo() const;
};
```

```
#endif


// --- Jugador.h ---
#ifndef JUGADOR_H
#define JUGADOR_H
#include <string>


class Jugador {
private:
    std::string alias;
    int rango;
    int puntajeFinal;


public:
    Jugador(std::string _alias);
    int calcularBonificacionRecursiva(int t);
    void procesarResultado(Partida p);
    std::string getAlias() const { return alias; }
    int getPuntaje() const { return puntajeFinal; }
    int getRango() const { return rango; }
};
#endif


// --- Ahorcado.h ---
#ifndef AHORCADO_H
```

```
#define AHORCADO_H

#include <string>

#include <vector>


class Ahorcado {

private:

    std::vector<std::string> palabrasNivel1;

    std::vector<std::string> palabrasNivel2;

    int intentosMaximos;

    bool esLetraValida(char c);


public:

    Ahorcado();

    void mostrarMenu();

    void iniciarJuego(std::vector<std::string> listaPalabras);

};

#endif
```

f) Casos de prueba

```
Palabra: _____
Intentos restantes: 3
Letras usadas: F T A
Ingrese una letra: E
>> Letra incorrecta. <<

--------------------------------
Palabra: _____
Intentos restantes: 2
Letras usadas: F T A E
Ingrese una letra: Ñ
>> Error: Solo se permiten letras (A-Z). <<

--------------------------------
Palabra: _____
Intentos restantes: 2
Letras usadas: F T A E
Ingrese una letra: L
>> Letra incorrecta. <<

--------------------------------
Palabra: _____
Intentos restantes: 1
Letras usadas: F T A E L
Ingrese una letra: V
>> Letra incorrecta. <<

GAME OVER. La palabra era: MISION

================================
   MODULO OBLIGATORIO: AHORCADO
================================
1. Jugar Nivel Inicial (Misiones)
2. Jugar Nivel Avanzado (Sistemas)
3. Ver Instrucciones
4. Volver al Menu Principal
Seleccione una opcion:
```

Ilustración 2. Caso de prueba 1

```
Palabra: _O_I_I_A_IO_
Intentos restantes: 2
Letras usadas: L E D U A I O
Ingrese una letra: b
>> ┬íBuen acierto! <<

--------------------------------
Palabra: BO_I_I_A_IO_
Intentos restantes: 2
Letras usadas: L E D U A I O B
Ingrese una letra: n
>> ┬íBuen acierto! <<

--------------------------------
Palabra: BONI_I_A_ION
Intentos restantes: 2
Letras usadas: L E D U A I O B N
Ingrese una letra: f
>> ┬íBuen acierto! <<

--------------------------------
Palabra: BONIFI_A_ION
Intentos restantes: 2
Letras usadas: L E D U A I O B N F
Ingrese una letra: c
>> ┬íBuen acierto! <<

┬íFELICIDADES! Ganaste la ronda. Palabra: BONIFICACION

================================
   MODULO OBLIGATORIO: AHORCADO
================================
1. Jugar Nivel Inicial (Misiones)
2. Jugar Nivel Avanzado (Sistemas)
3. Ver Instrucciones
4. Volver al Menu Principal
Seleccione una opcion:
```

Ilustración 3. Caso de prueba 2

```
Palabra: ____O_
Intentos restantes: 5
Letras usadas: E O
Ingrese una letra: i
>> ┬íBuen acierto! <<

--------------------------------
Palabra: _I_IO_
Intentos restantes: 5
Letras usadas: E O I
Ingrese una letra: m
>> ┬íBuen acierto! <<

--------------------------------
Palabra: MI_IO_
Intentos restantes: 5
Letras usadas: E O I M
Ingrese una letra: s
>> ┬íBuen acierto! <<

--------------------------------
Palabra: MISIO_
Intentos restantes: 5
Letras usadas: E O I M S
Ingrese una letra: n
>> ┬íBuen acierto! <<

┬íFELICIDADES! Ganaste la ronda. Palabra: MISION

================================
   MODULO OBLIGATORIO: AHORCADO
================================
1. Jugar Nivel Inicial (Misiones)
2. Jugar Nivel Avanzado (Sistemas)
3. Ver Instrucciones
4. Volver al Menu Principal
Seleccione una opcion:
```

Ilustración 4. Caso de prueba 3

2.7 Resultados obtenidos

De acuerdo con el tipo de trabajo, se plasmarán los resultados alcanzados en la guía práctica una vez ejecutadas las actividades. Se pueden emplear figuras y tablas las cuales deben ser numeradas.

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☒ Trabajo en equipo
☐ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad

☒ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

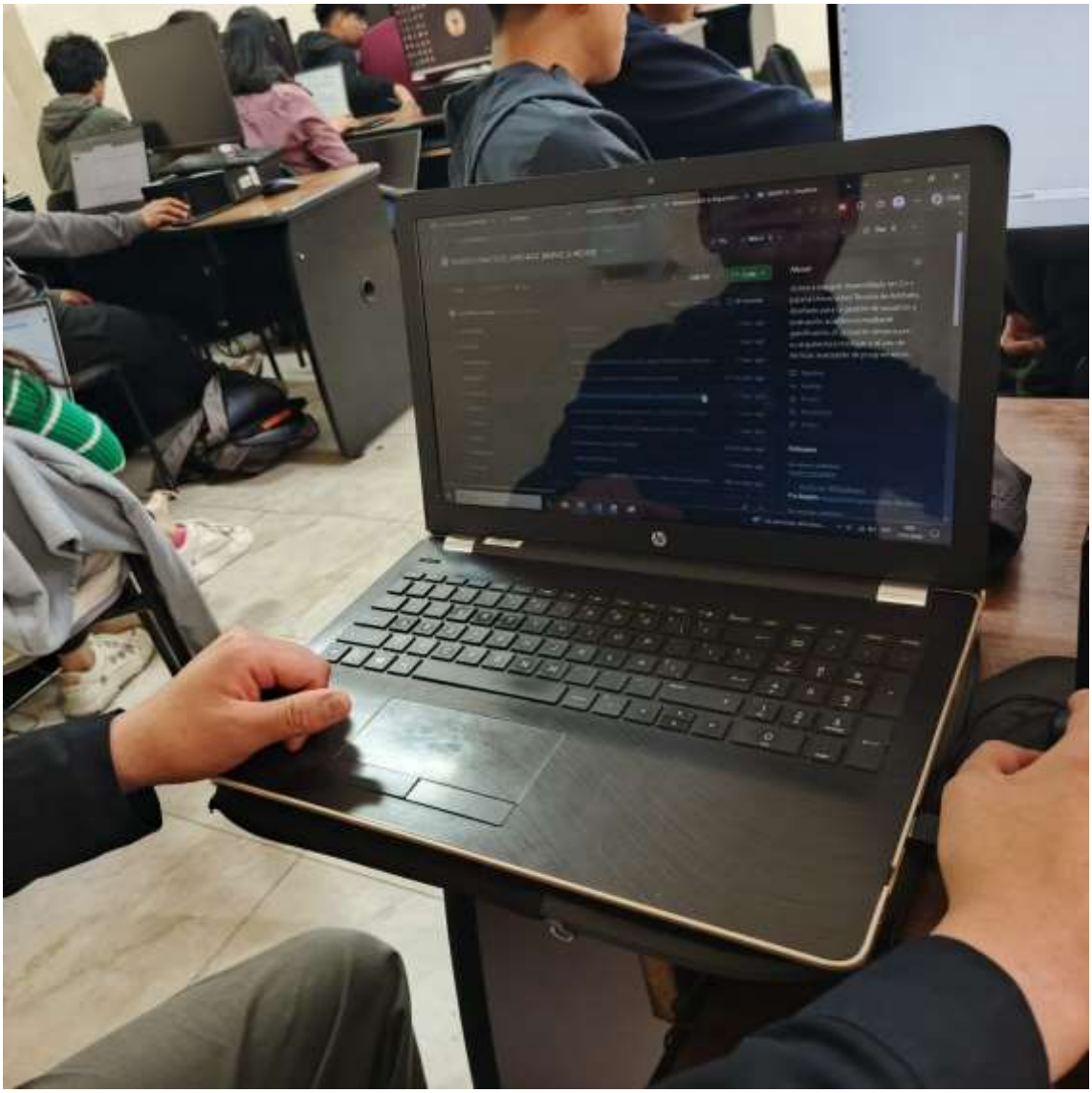
2.9 Conclusiones

- Se logró una estructura de software organizada y escalable mediante la separación de responsabilidades en las clases Jugador, Partida y Ahorcado, demostrando que el **encapsulamiento** y la modularidad facilitan el mantenimiento del código y protegen la integridad de los datos del videojuego.
- El uso de **GitHub Desktop** como herramienta de control de versiones permitió registrar el progreso incremental del proyecto. El historial de commits generado sirve como evidencia de una metodología de trabajo profesional y asegura la trazabilidad de cada cambio realizado en el sistema.
- La implementación de la **función recursiva** para el cálculo de bonificaciones permitió procesar puntajes de manera técnica y eficiente. Asimismo, la depuración del código defectuoso en la lógica de rangos garantizó un sistema de clasificación justo y sin errores lógicos de redundancia.

2.10Recomendaciones

- Se recomienda implementar capas de validación más estrictas en la clase Partida, utilizando bloques try-catch o verificaciones adicionales para asegurar que los puntos de las misiones y el tiempo ingresado siempre se mantengan dentro de rangos lógicos (ej. valores no negativos).
- Para futuras versiones académicas, se sugiere integrar el módulo de Ahorcado con una **base de datos o archivos externos (.txt)**, permitiendo que el vocabulario de misiones y penalizaciones pueda actualizarse sin necesidad de modificar y recompilar el código fuente.
- Es aconsejable mantener el uso de **ramas (branches)** en GitHub para probar nuevas funcionalidades (como un ranking global en la nube) antes de integrarlas a la rama principal (main), garantizando así que la versión estable del videojuego nunca se vea comprometida durante el desarrollo.

2.11 Referencias bibliográficas

[1] I. Bravo Sánchez, “EXAMEN_PRACTICO_UNIDAD2_BRAVO_S-NCHEZ,” GitHub, 2026. [En línea]. Disponible en: https://github.com/isma1402uta-boop/EXAMEN_PRACTICO_UNIDAD2_BRAVO_S-NCHEZ/issues. [Accedido: 15-abr-2026].

2.12 Anexos